

IRIS DATASET VISUALIZATION(SEABORN,MATPLOTLIB)

plotting graph with seaborn

```
In [1]: import numpy as np #linear algebra
import pandas as pd #data processing, CSV file I/O (i.e.pd.read_csv)
```

Importing pandas and Seaborn module

```
In [2]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt #plt.style.use('fivethirtyeight')
import warnings
warnings.filterwarnings('ignore') #this will ignore the warnings.it wont displa
```

```
In [3]: iris = pd.read_csv(r"C:\Users\Ramya\Downloads\24th, 25th- Advanced EDA project\2
```

```
In [4]: iris
```

Out[4]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

In [5]: `iris.head() #displaying data`

Out[5]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

In [6]: `iris.drop('Id', axis = 1,inplace = True)`In [7]: `iris.head()`

Out[7]:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

In [8]: `iris.info()`

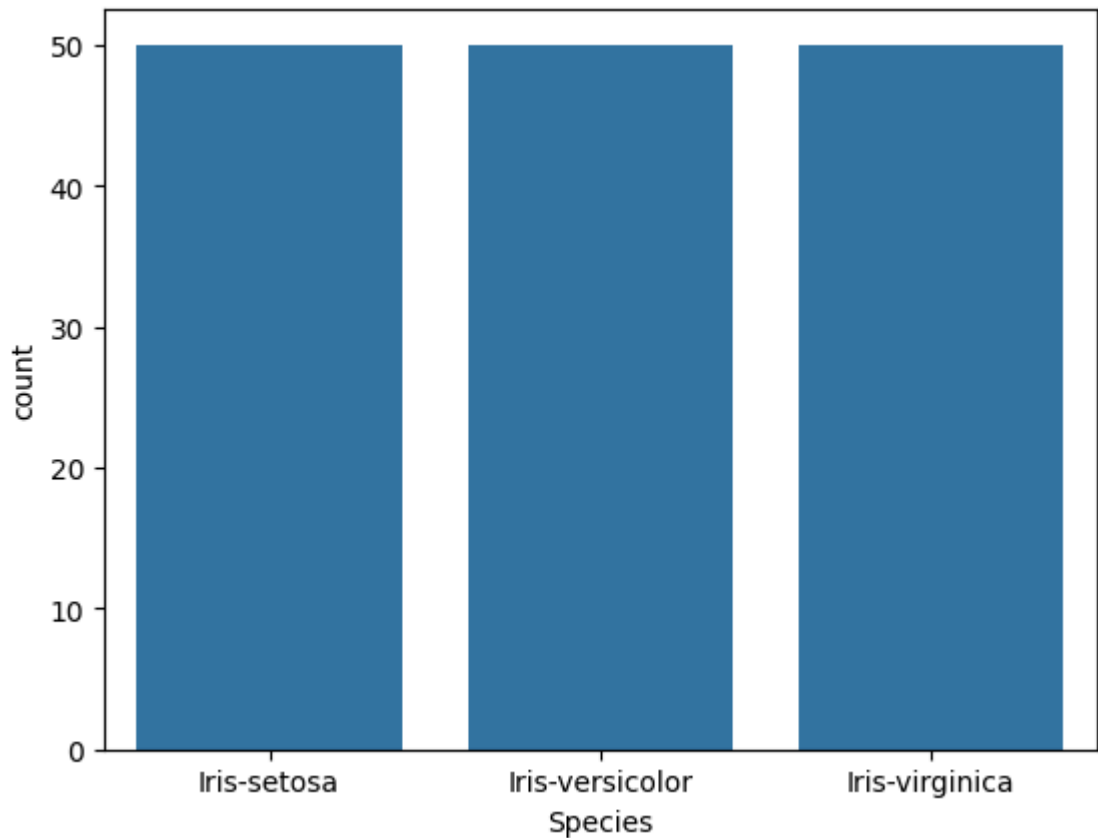
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   SepalLengthCm    150 non-null    float64
1   SepalWidthCm     150 non-null    float64
2   PetalLengthCm    150 non-null    float64
3   PetalWidthCm     150 non-null    float64
4   Species          150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

In [9]: `iris['Species'].value_counts()`

Out[9]:

```
Species
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: count, dtype: int64
```

In [10]: `sns.countplot(x='Species',data=iris) #this data set has 3 varieties of iris plant`
`plt.show() #Bar plot: plotting frequency of the observation`

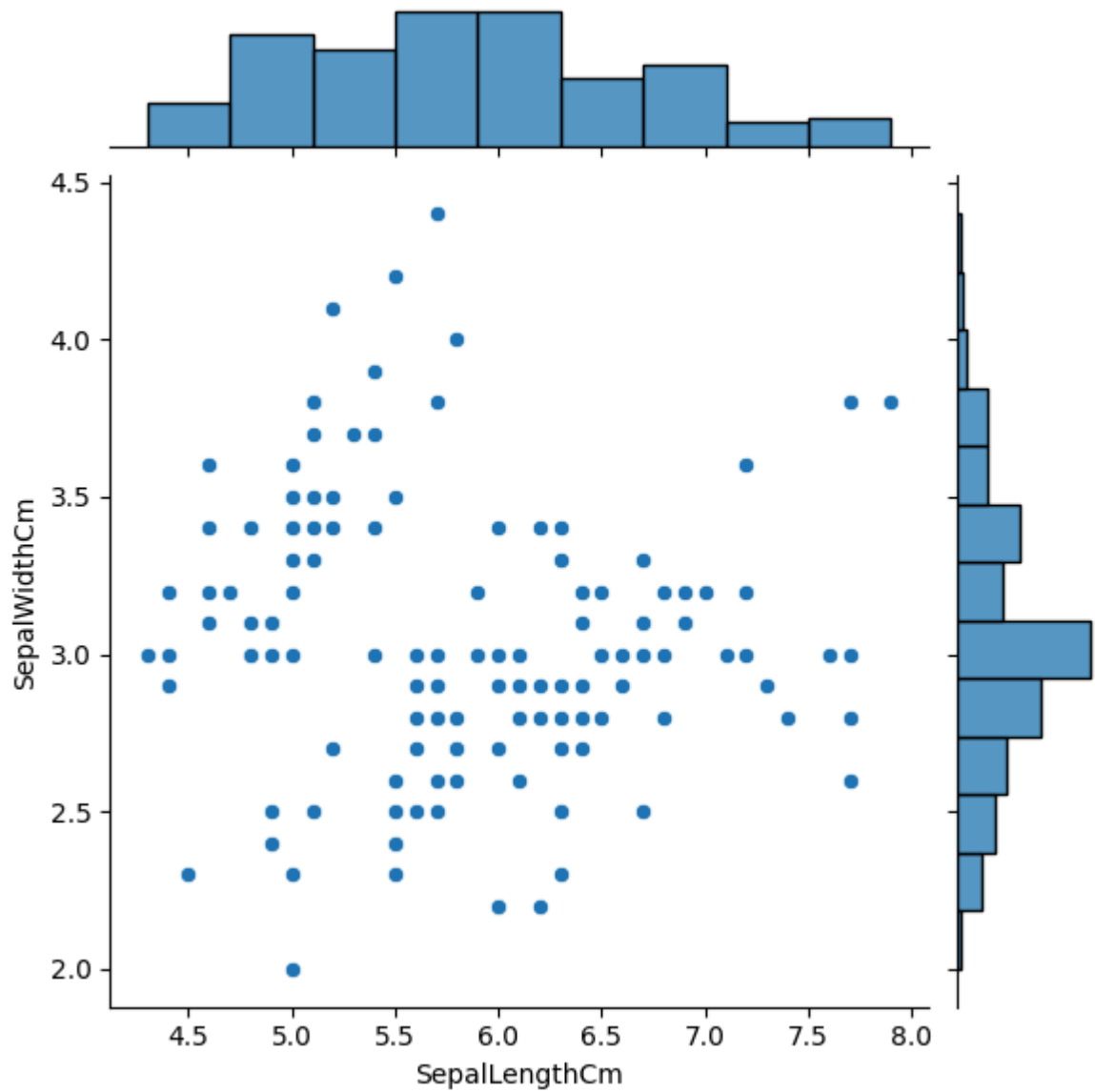


```
In [11]: iris.head() #50 samples each of all the iris Species in the dataset
```

```
Out[11]:
```

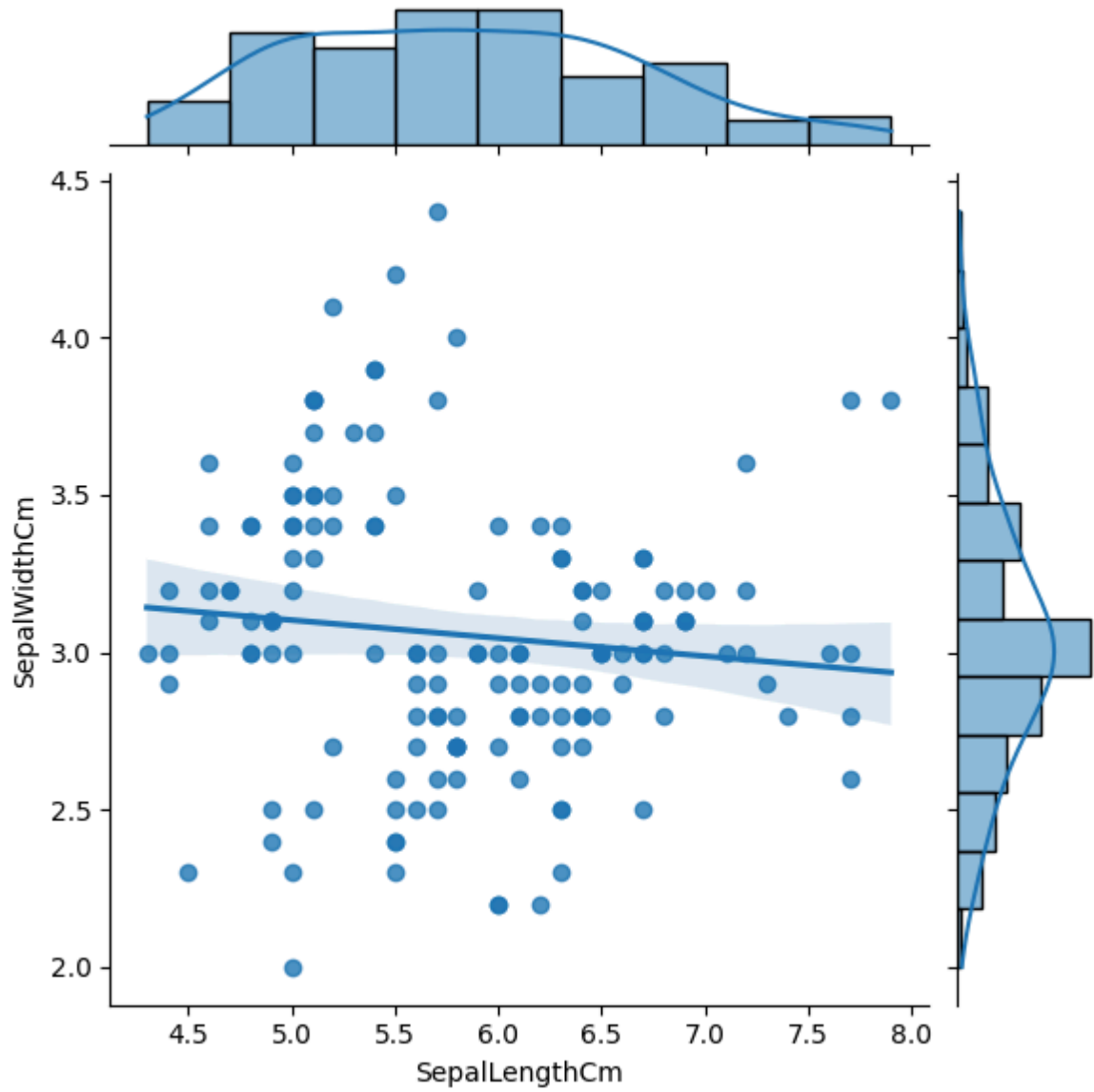
	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [12]: fig = sns.jointplot(x='SepalLengthCm', y = 'SepalWidthCm', data = iris)
```

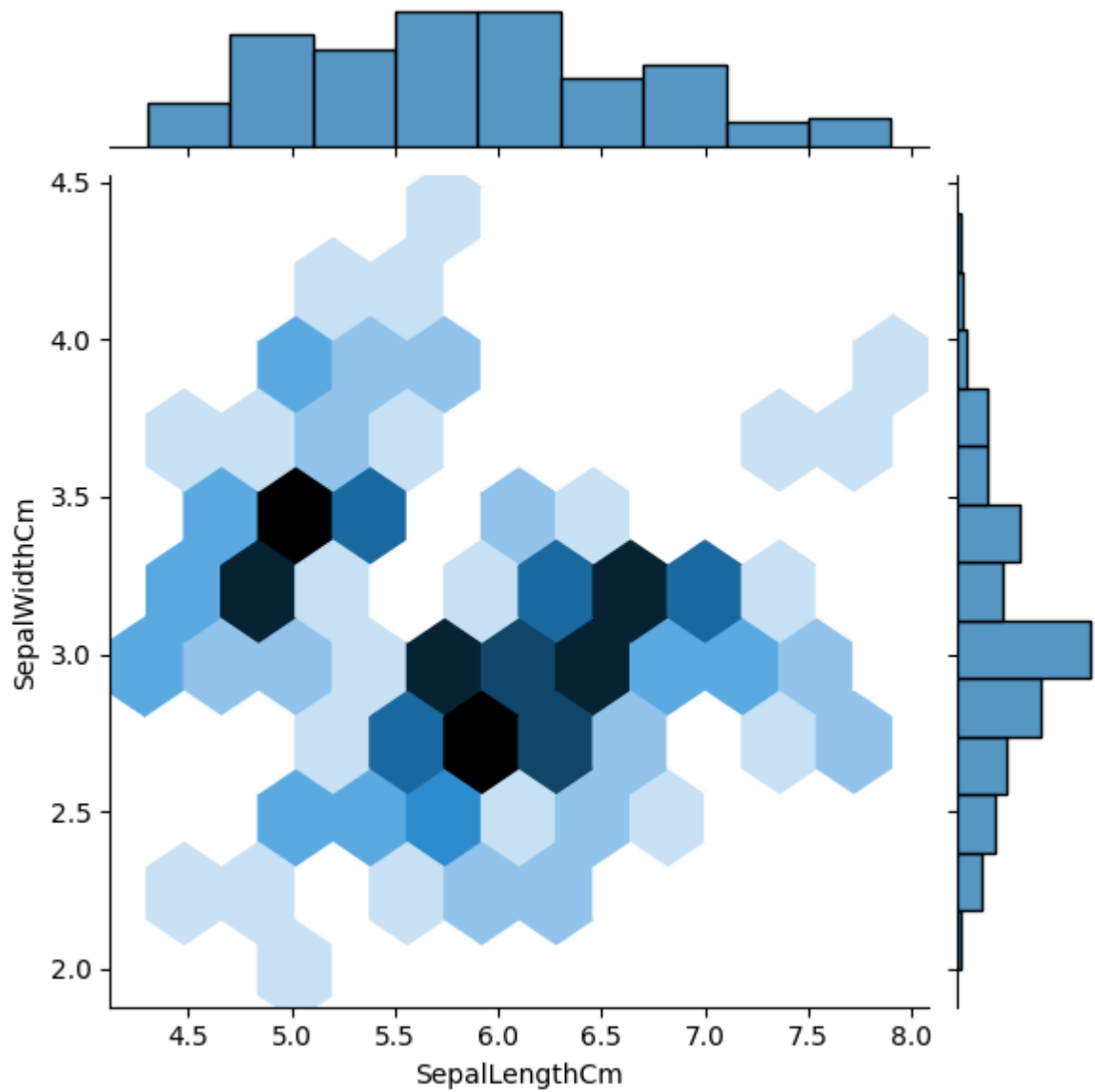


```
In [13]: sns.jointplot(x="SepalLengthCm", y="SepalWidthCm", data=iris, kind="reg")
```

```
Out[13]: <seaborn.axisgrid.JointGrid at 0x1801ed99790>
```

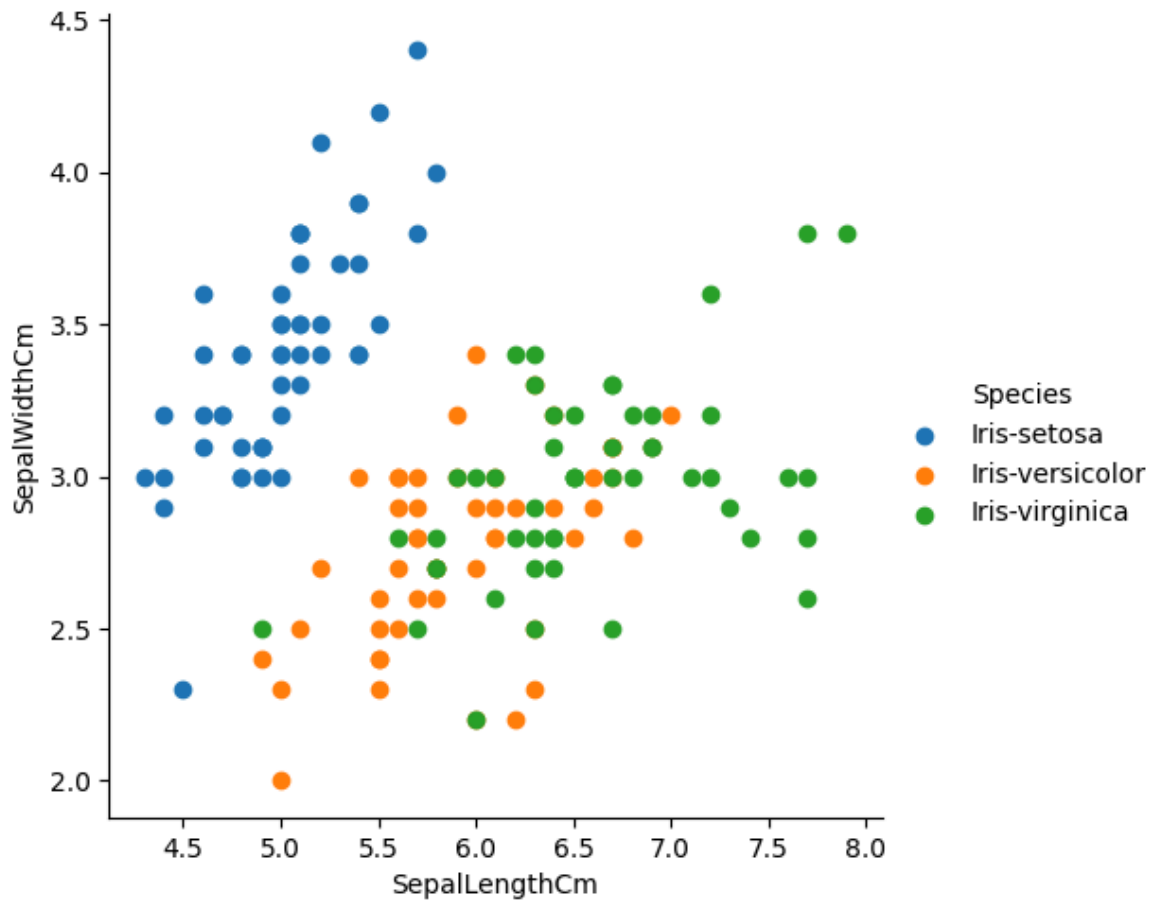


```
In [14]: fig=sns.jointplot(x='SepalLengthCm',y='SepalWidthCm',kind='hex',data=iris)
```



```
In [15]: import matplotlib.pyplot as plt
%matplotlib inline
# FacetGrid plot

sns.FacetGrid(iris, hue='Species', height=5)\
.map(plt.scatter, 'SepalLengthCm', 'SepalWidthCm')\
.add_legend()
plt.show()
```



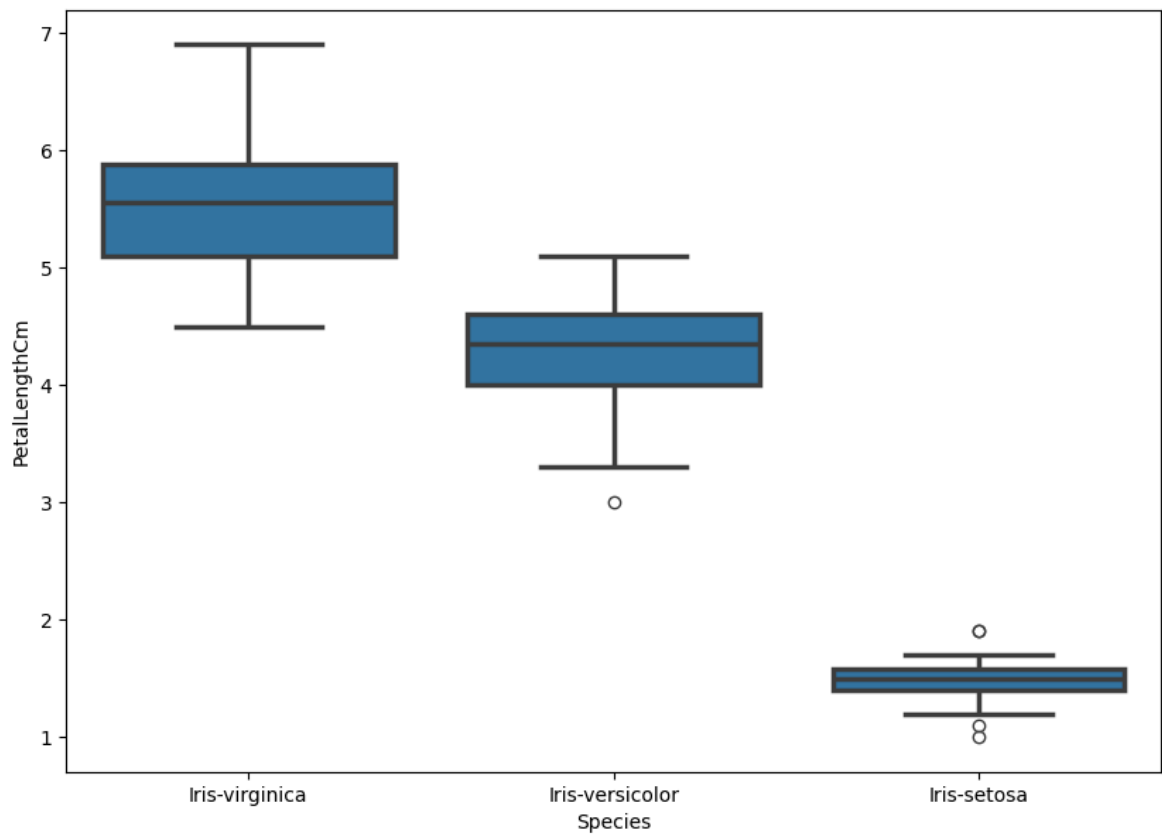
In [16]: `iris.head()` #Box plot or whisker plot

Out[16]:

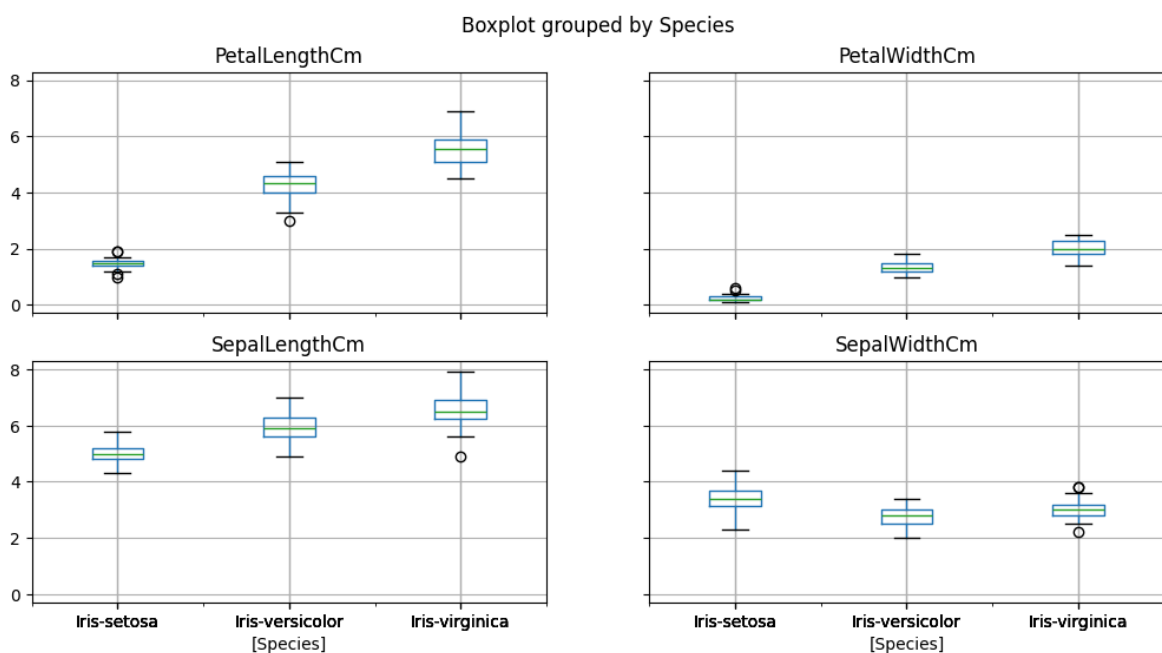
	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

In [17]: `fig=plt.gcf()`
`fig.set_size_inches(10,7)`
`fig=sns.boxplot(x='Species',y='PetalLengthCm',data=iris,order=['Iris-virginica',`

In [18]: `plt.show()`

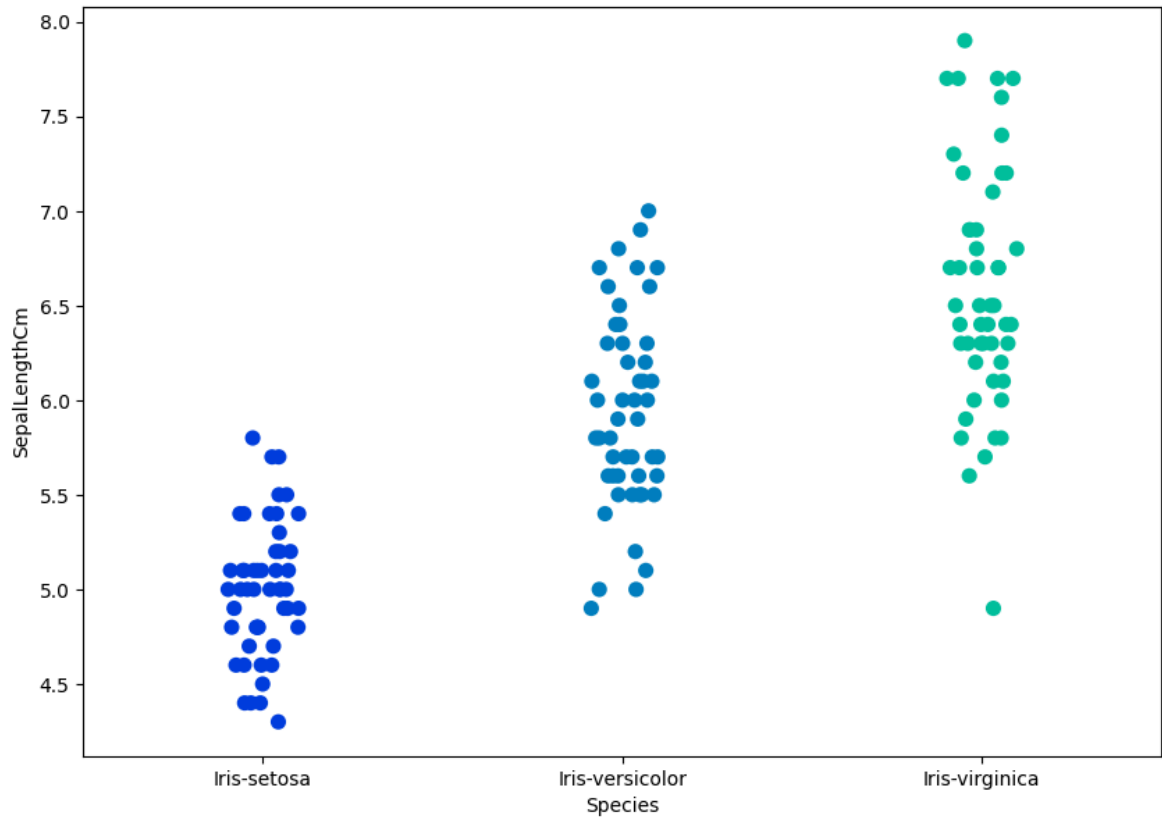


```
In [19]: iris.boxplot(by="Species", figsize=(12, 6))
plt.show()
```



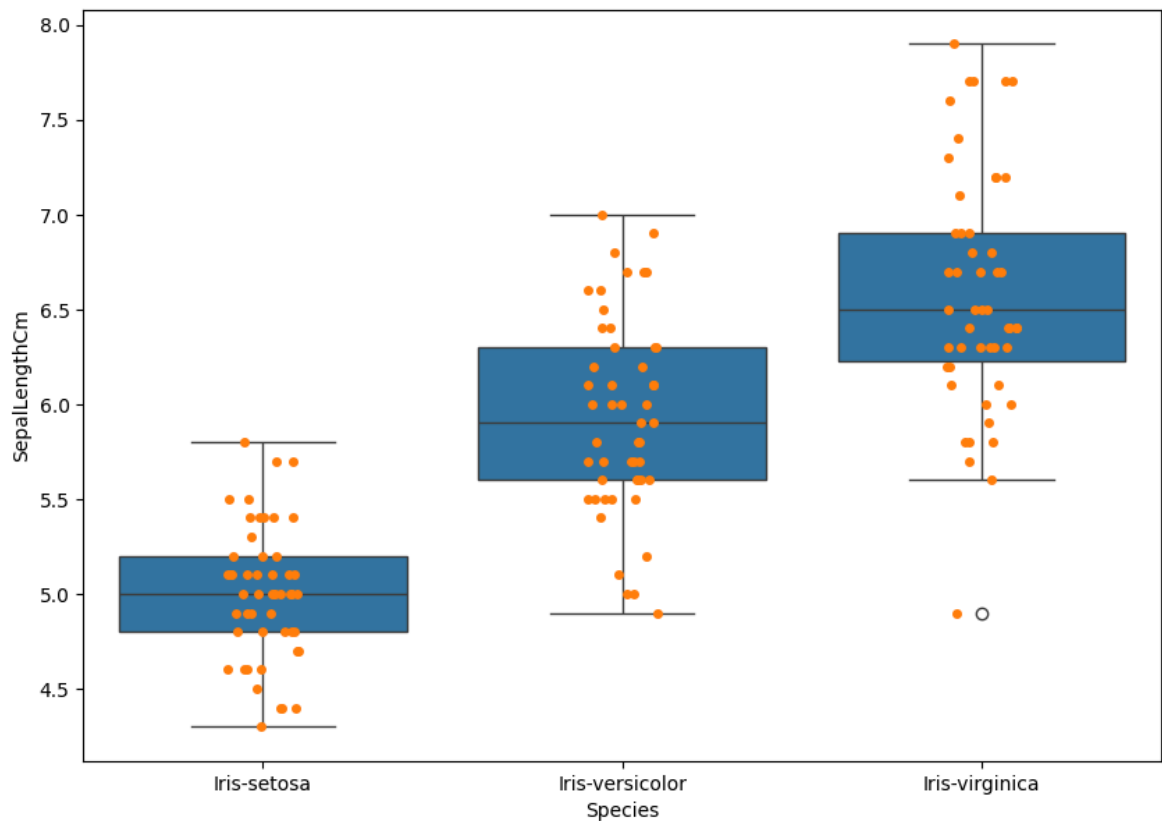
```
In [20]: fig=plt.gcf() #Strip plot
fig.set_size_inches(10,7)
fig=sns.stripplot(x='Species',y='SepalLengthCm',data=iris,jitter=True,edgecolor=
```

```
In [21]: plt.show()
```



```
In [22]: fig=plt.gcf() #combing box and strip plot
fig.set_size_inches(10,7)
fig=sns.boxplot(x='Species',y='SepalLengthCm',data=iris)
fig=sns.stripplot(x='Species',y='SepalLengthCm',data=iris,jitter=True,edgecolor=
```

```
In [23]: plt.show()
```



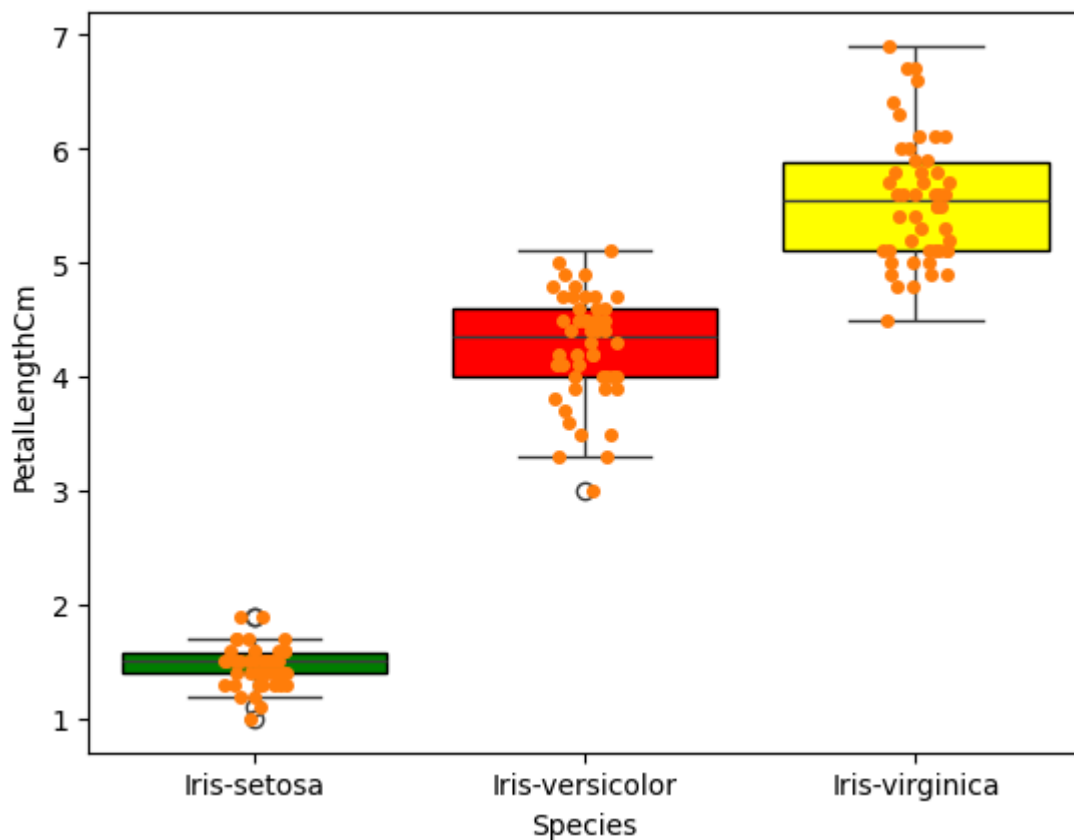
```
In [24]: import seaborn as sns
import matplotlib.pyplot as plt

# Create boxplot and overlay stripplot
ax = sns.boxplot(x="Species", y="PetalLengthCm", data=iris)
sns.stripplot(x="Species", y="PetalLengthCm", data=iris, jitter=True, edgecolor=

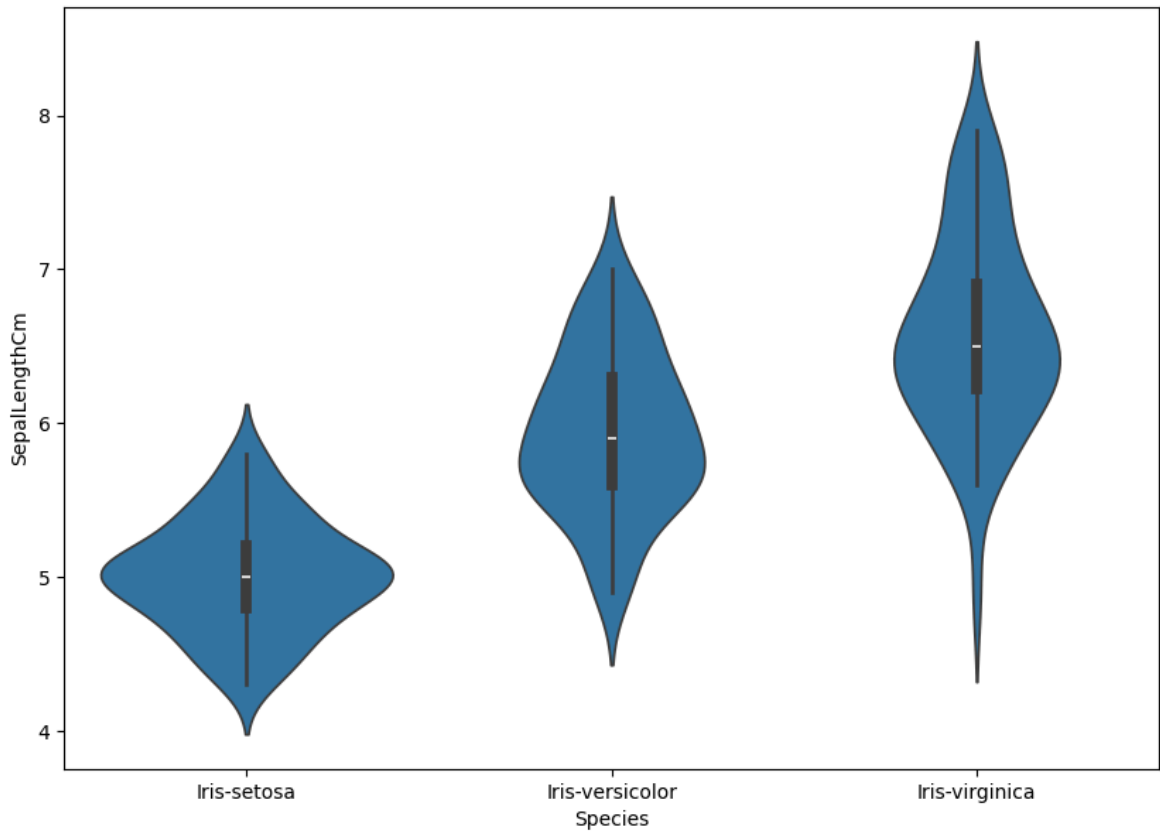
# Change the colors of the three boxes
colors = ['green', 'red', 'yellow']

# Each box is made of a Patch; they appear in ax.patches in order
for i, patch in enumerate(ax.patches[:3]): # only first 3 patches are the boxes
    patch.set_facecolor(colors[i])
    patch.set_edgecolor('black')

plt.show()
```



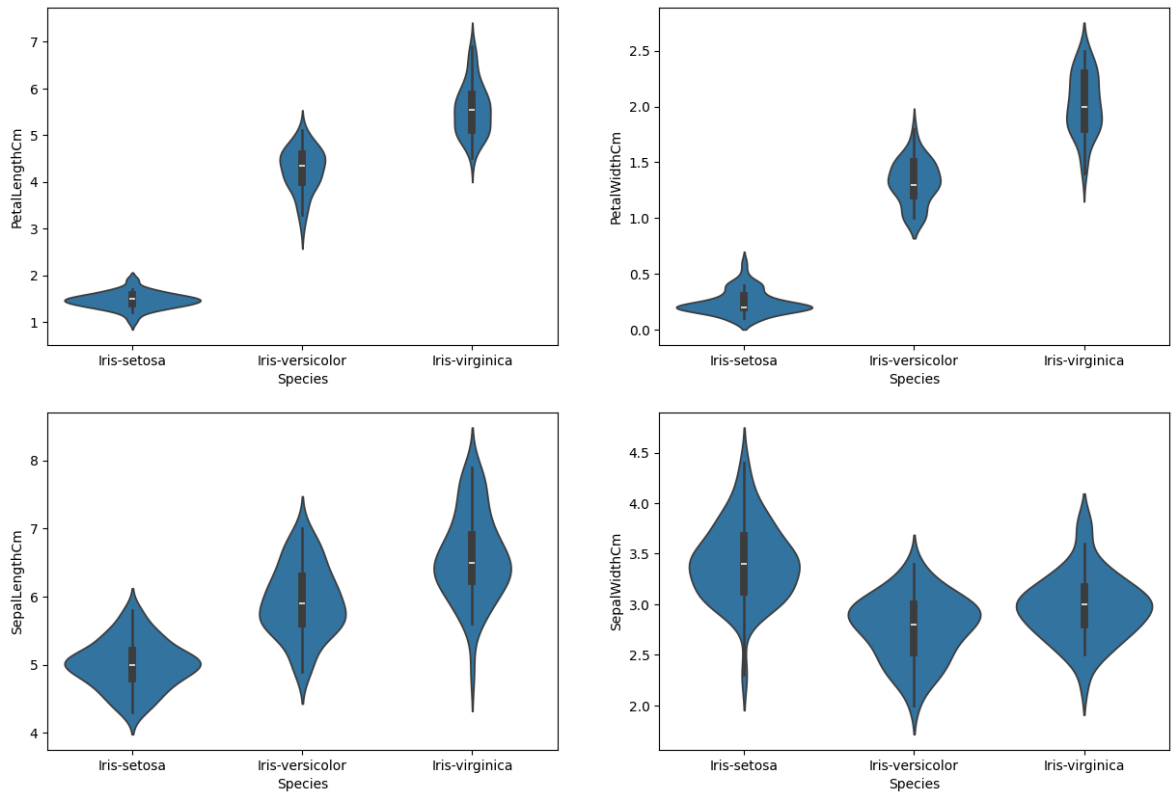
```
In [25]: fig=plt.gcf() #violin plot
fig.set_size_inches(10,7)
fig=sns.violinplot(x='Species',y='SepalLengthCm',data=iris)
plt.show()
```



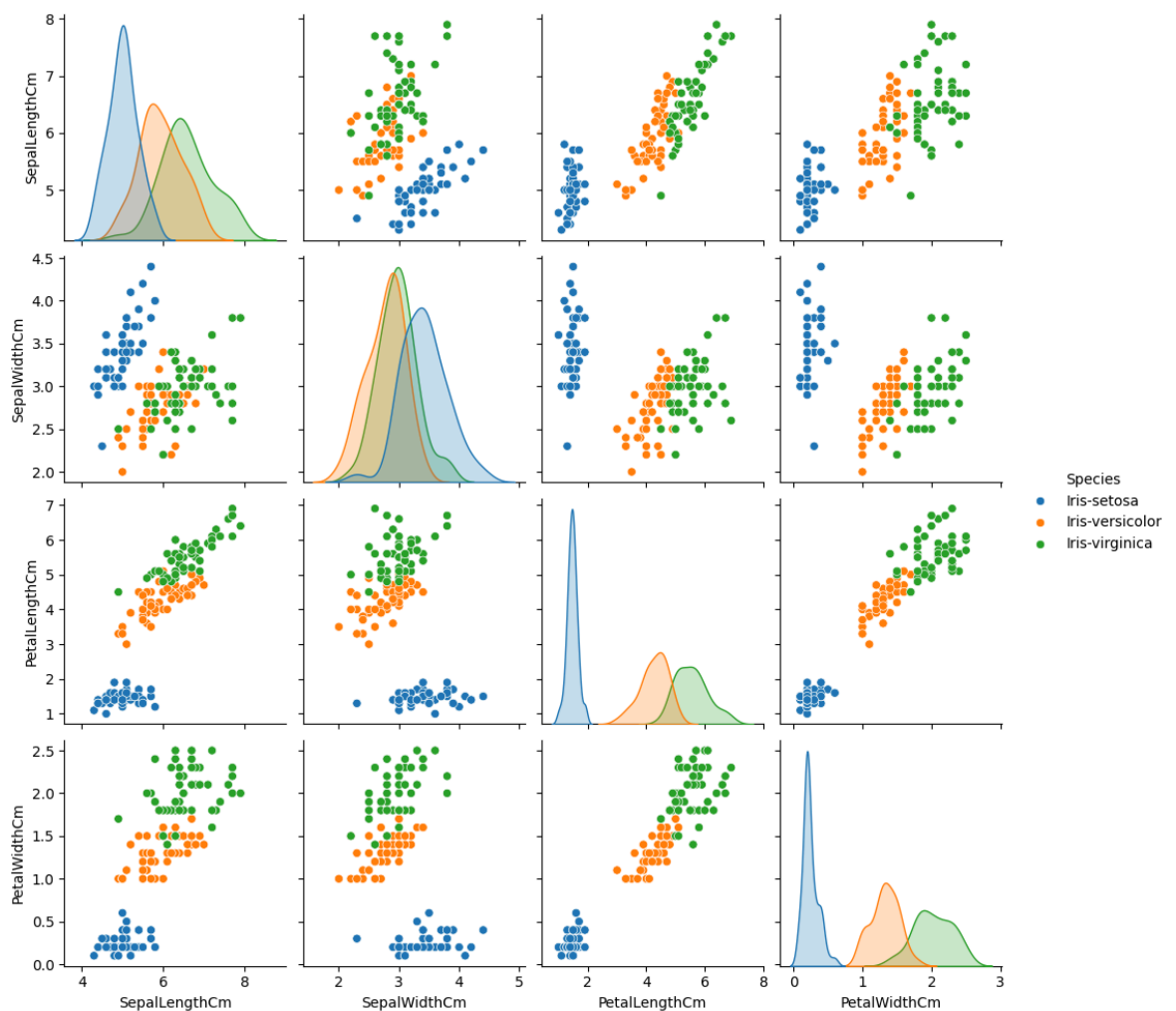
```
In [26]: plt.figure(figsize=(15,10))
plt.subplot(2,2,1)
sns.violinplot(x='Species',y='PetalLengthCm',data=iris)
plt.subplot(2,2,2)
sns.violinplot(x='Species',y='PetalWidthCm',data=iris)
plt.subplot(2,2,3)
sns.violinplot(x='Species',y='SepalLengthCm',data=iris)
plt.subplot(2,2,4)
sns.violinplot(x='Species',y='SepalWidthCm',data=iris)
```

```
Out[26]: <Axes: xlabel='Species', ylabel='SepalWidthCm'>
```

```
In [27]: plt.show()
```



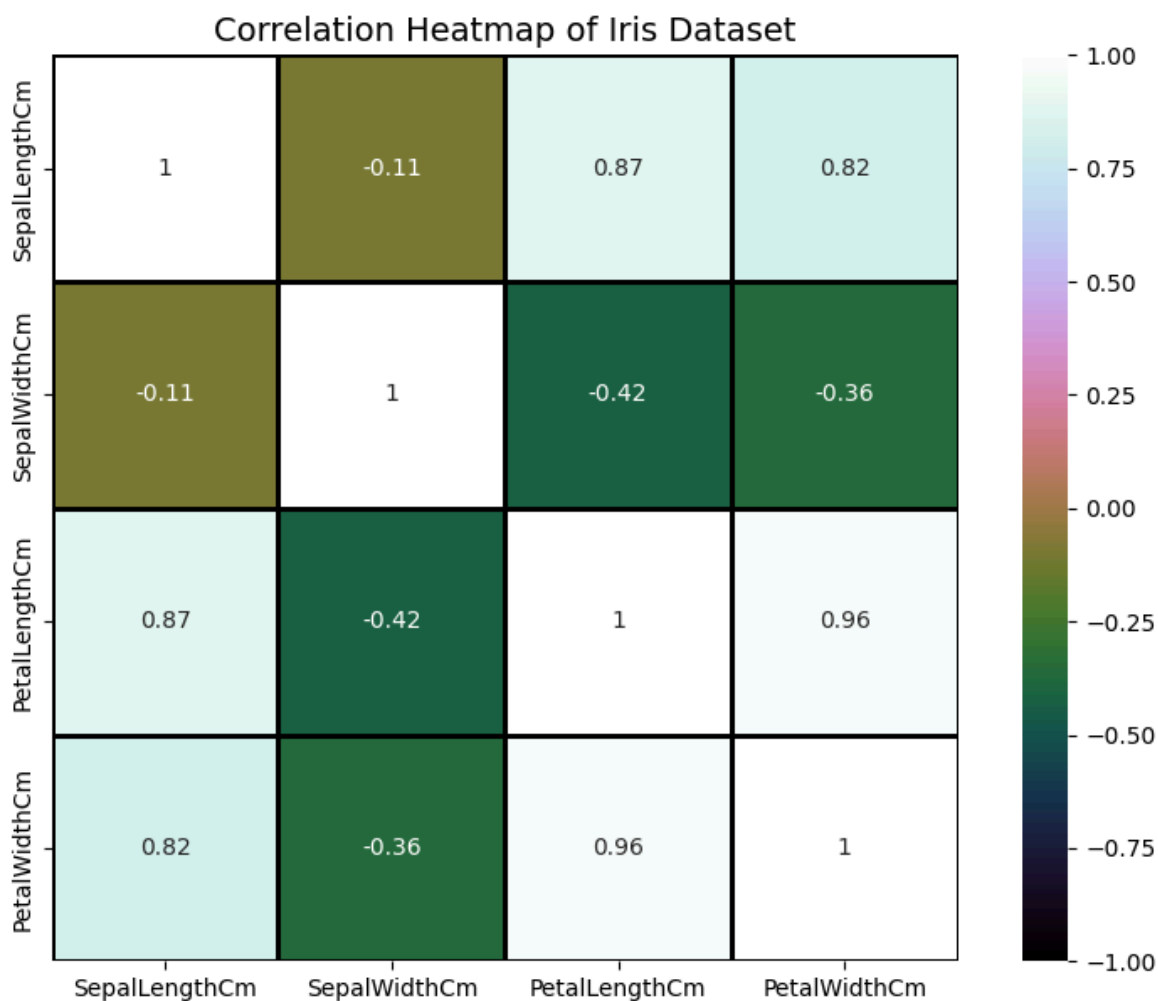
```
In [28]: sns.pairplot(iris,hue='Species'); # pairplot  
plt.show()
```



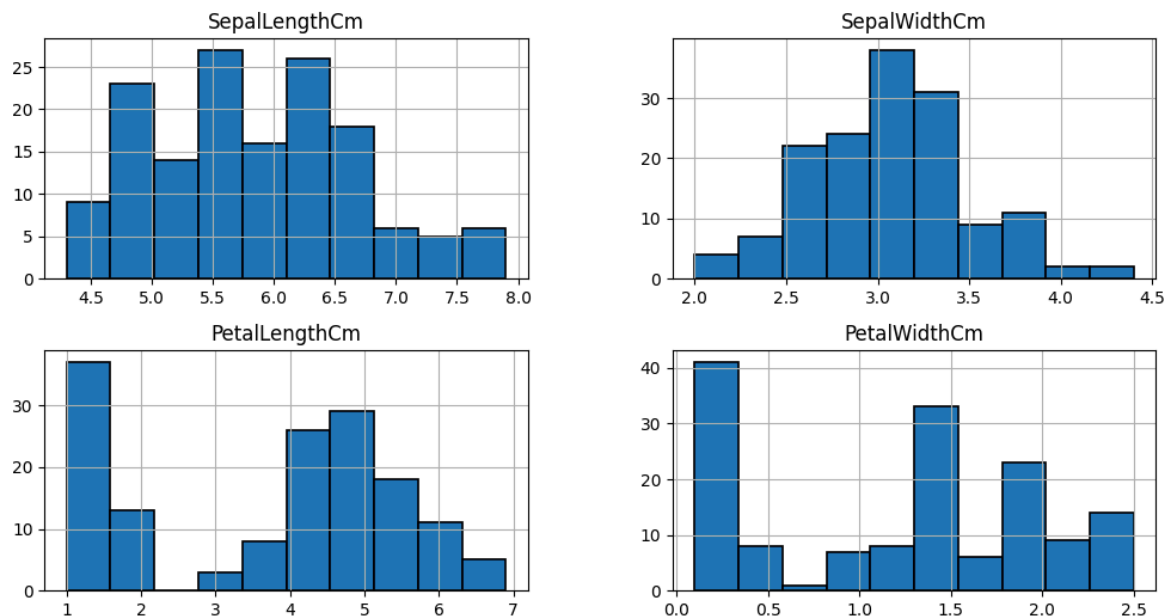
```
In [29]: # Get only numeric columns
numeric_iris = iris.select_dtypes(include='number') #Heat map

# Create figure and heatmap
fig = plt.figure(figsize=(10, 7))
sns.heatmap(numeric_iris.corr(), annot=True, cmap='cubehelix',
            linewidths=1, linecolor='k', square=True,
            vmin=-1, vmax=1,
            cbar_kws={"orientation": "vertical"}, cbar=True)

plt.title("Correlation Heatmap of Iris Dataset", fontsize=14)
plt.show()
```

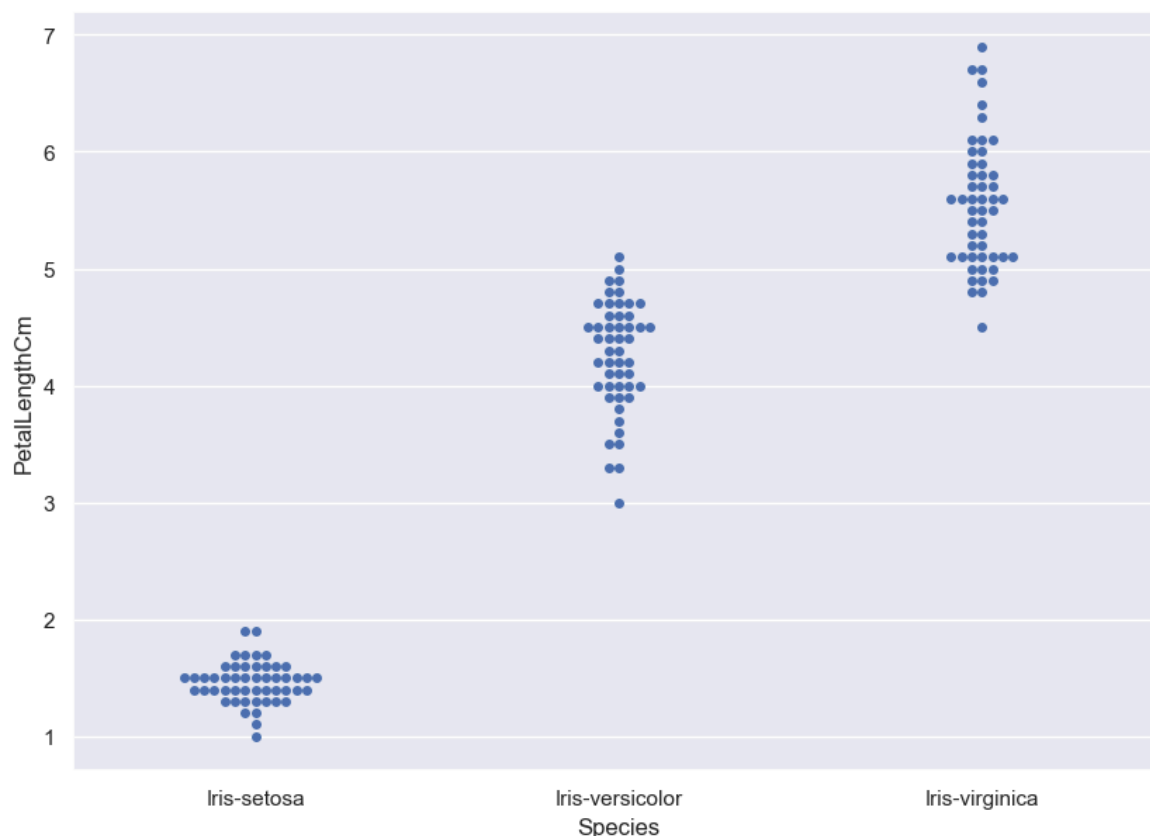


```
In [30]: iris.hist(edgecolor='black', linewidth=1.2) # Dist plot
fig=plt.gcf()
fig.set_size_inches(12,6)
plt.show()
```

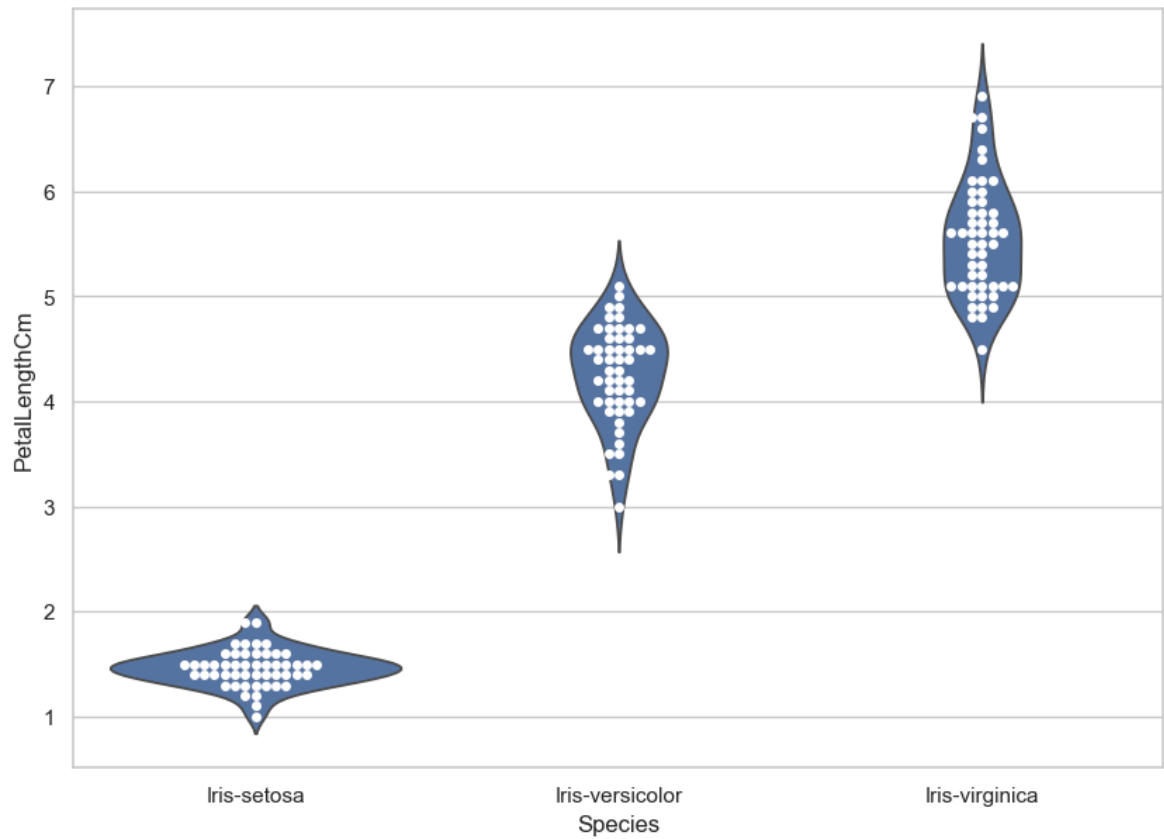


```
In [31]: sns.set(style="darkgrid") #Swarm plot
fig=plt.gcf()
fig.set_size_inches(10,7)
fig = sns.swarmplot(x="Species", y="PetalLengthCm", data=iris)
```

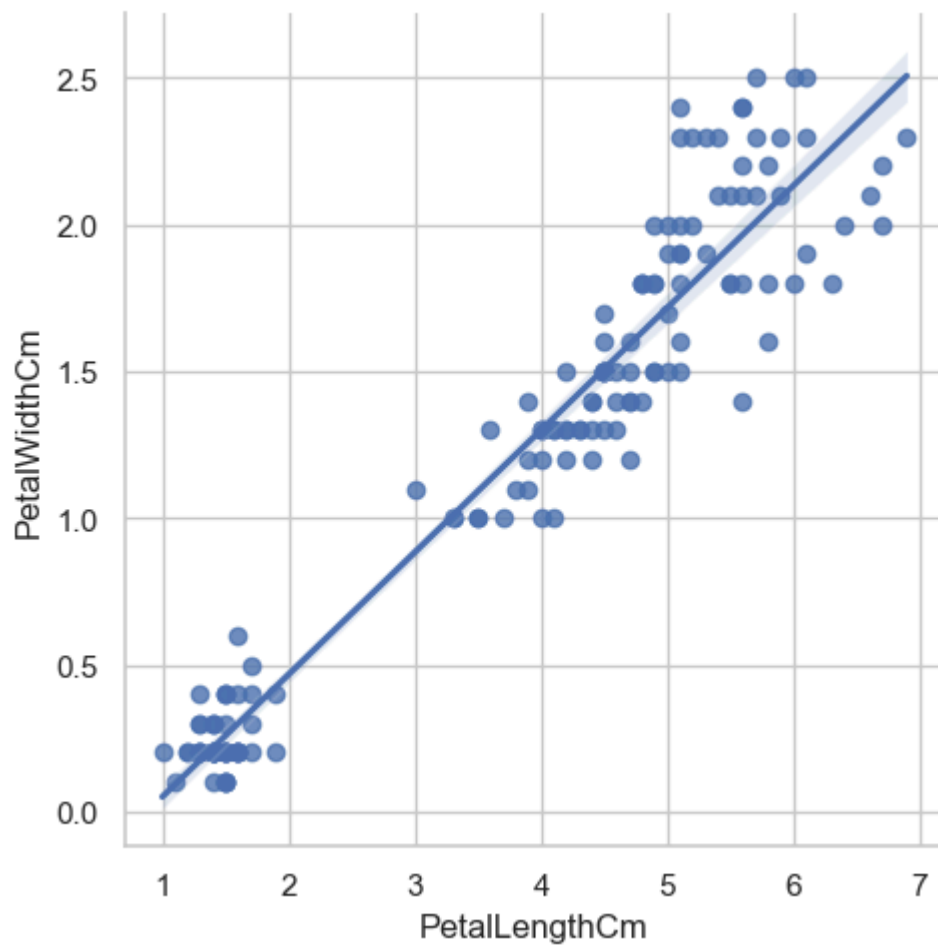
```
In [32]: plt.show()
```



```
In [33]: sns.set(style="whitegrid")
fig=plt.gcf()
fig.set_size_inches(10,7)
ax = sns.violinplot(x="Species", y="PetalLengthCm", data=iris, inner=None)
ax = sns.swarmplot(x="Species", y="PetalLengthCm", data=iris,color="white", edge
plt.show()
```



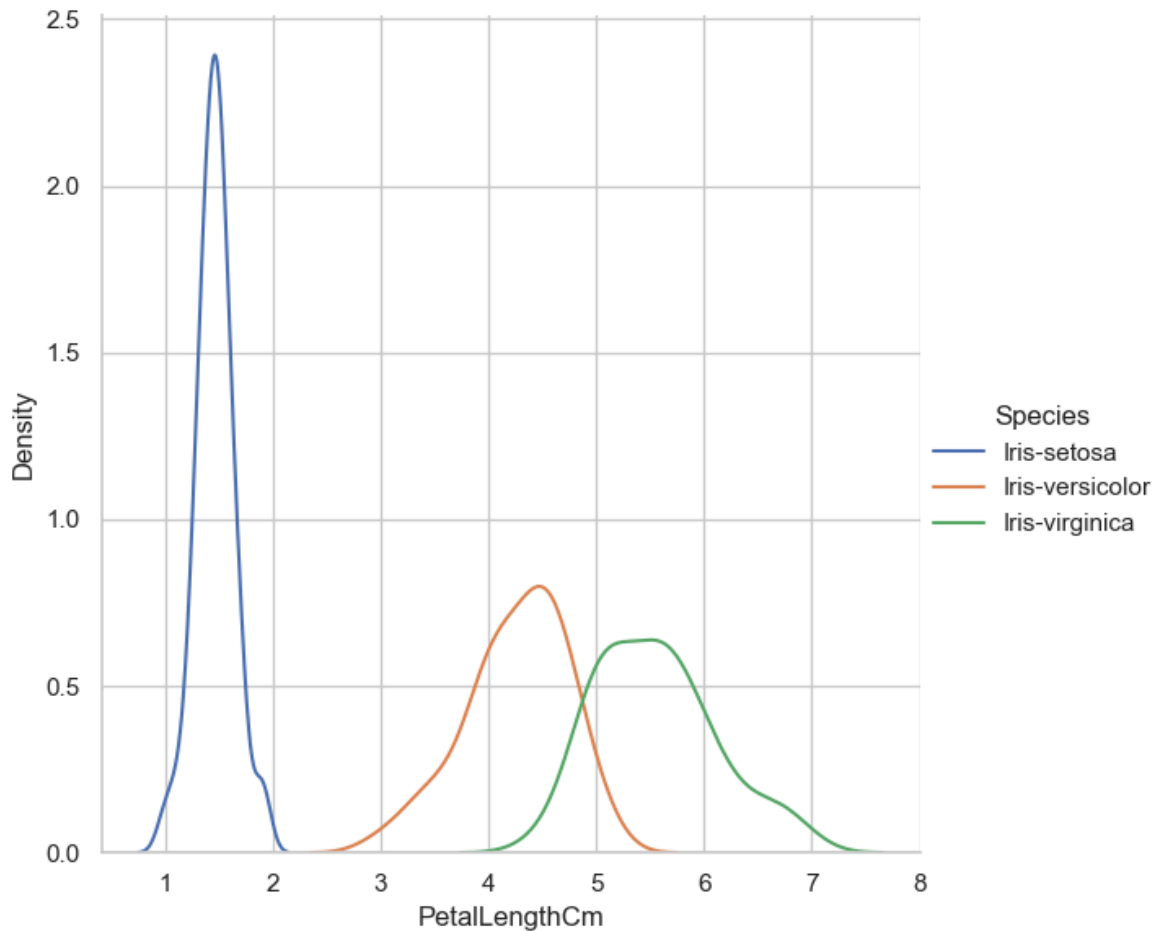
```
In [34]: fig=sns.lmplot(x="PetalLengthCm", y="PetalWidthCm",data=iris) #lm plot  
plt.show()
```



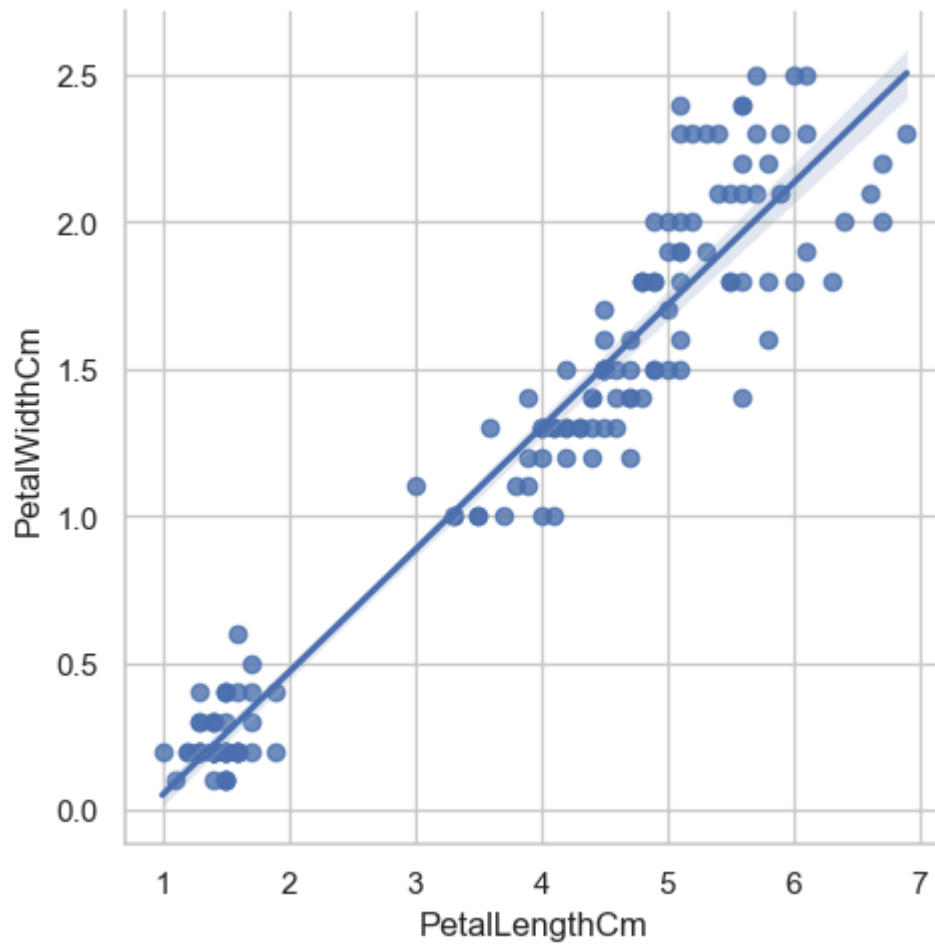

```
In [35]: sns.FacetGrid(iris, hue="Species", height=6) \
        .map(sns.kdeplot, "PetalLengthCm") \
        .add_legend()
plt.ioff() #FacetGrid
```

Out[35]: <contextlib.ExitStack at 0x180232ee3f0>

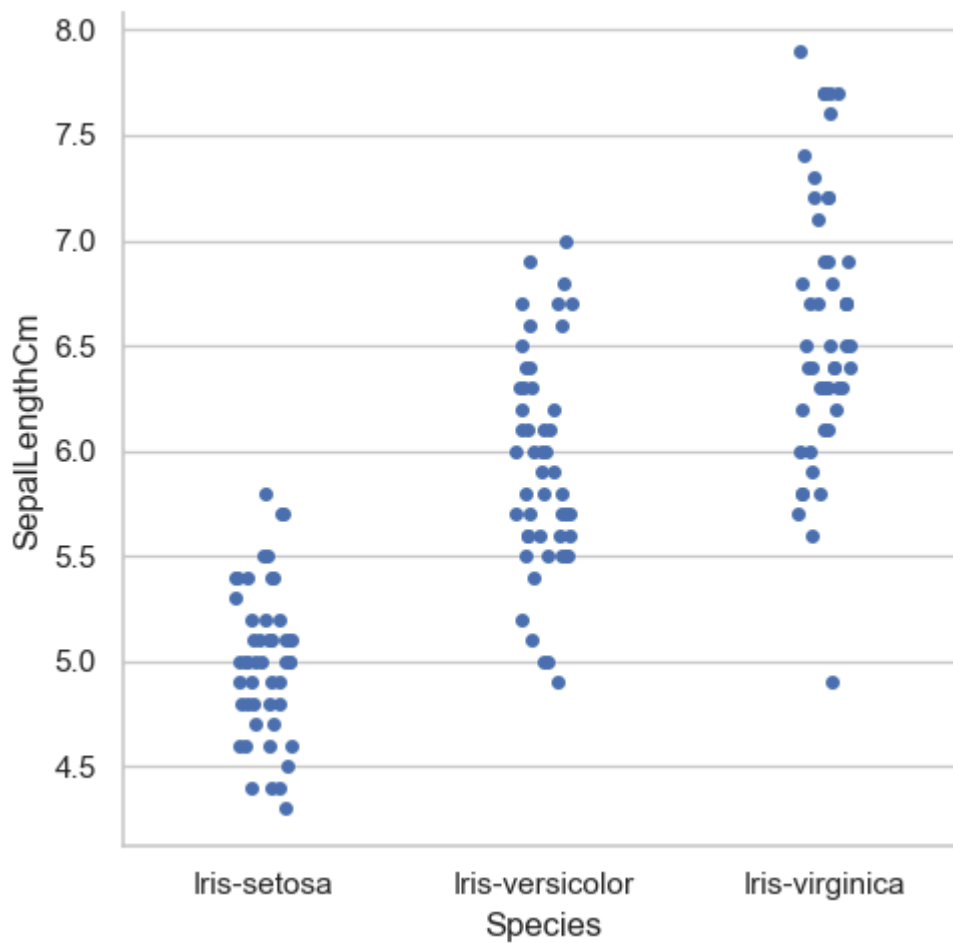
```
In [36]: plt.show()
```



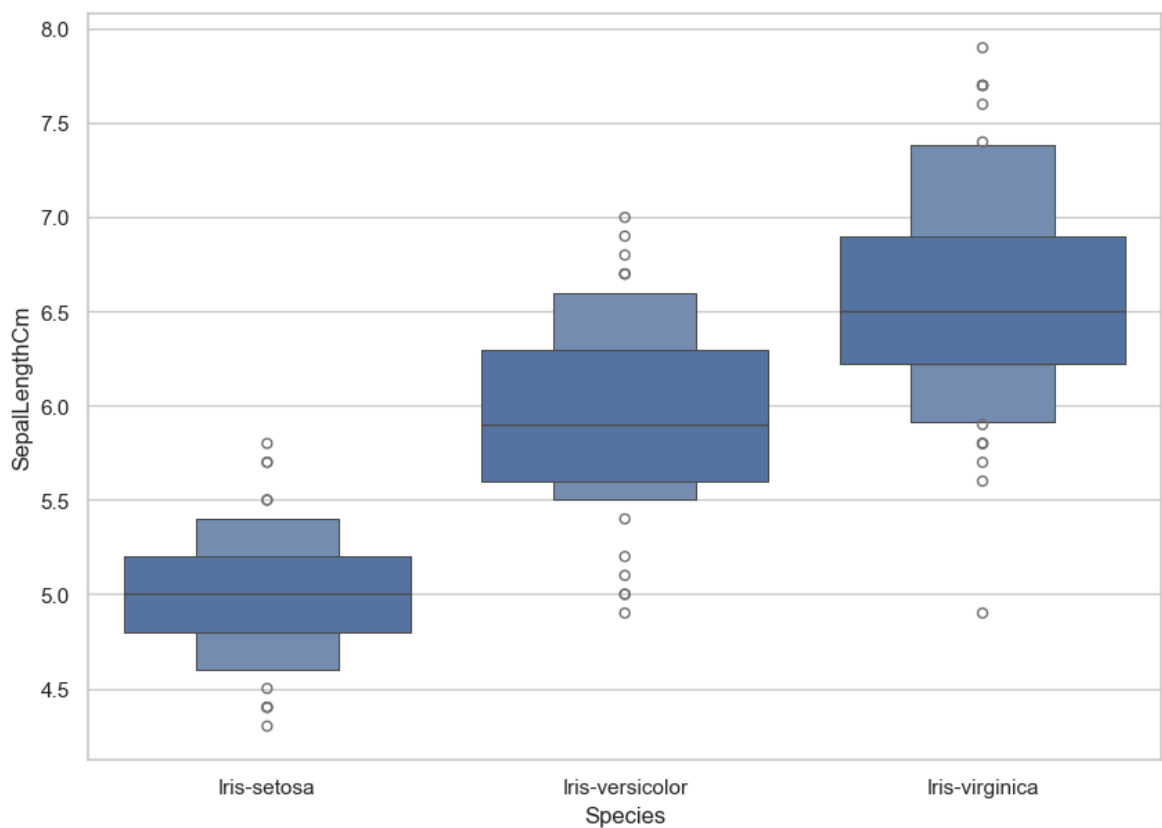
```
In [37]: fig=sns.lmplot(x="PetalLengthCm", y="PetalWidthCm",data=iris)
plt.show()
```



```
In [38]: sns.catplot(x='Species',y='SepalLengthCm',data=iris)
plt.ioff()
plt.show() #Factor plot ot cat plot
```



```
In [39]: fig=plt.gcf() #boxenplot
fig.set_size_inches(10,7)
fig=sns.boxenplot(x='Species',y='SepalLengthCm',data=iris)
plt.show()
```



```
In [43]: # Create a kde plot of sepal_length versus sepal width for setosa species of flo
sub=iris[iris['Species']=='Iris-setosa'] #kde plot
sns.kdeplot(data=sub[[x='SepalLengthCm',y='SepalWidthCm']],cmap="plasma", shade=
plt.title('Iris-setosa')
plt.xlabel('Sepal Length Cm')
plt.ylabel('Sepal Width Cm')
plt.show()
```

Cell In[43], line 3

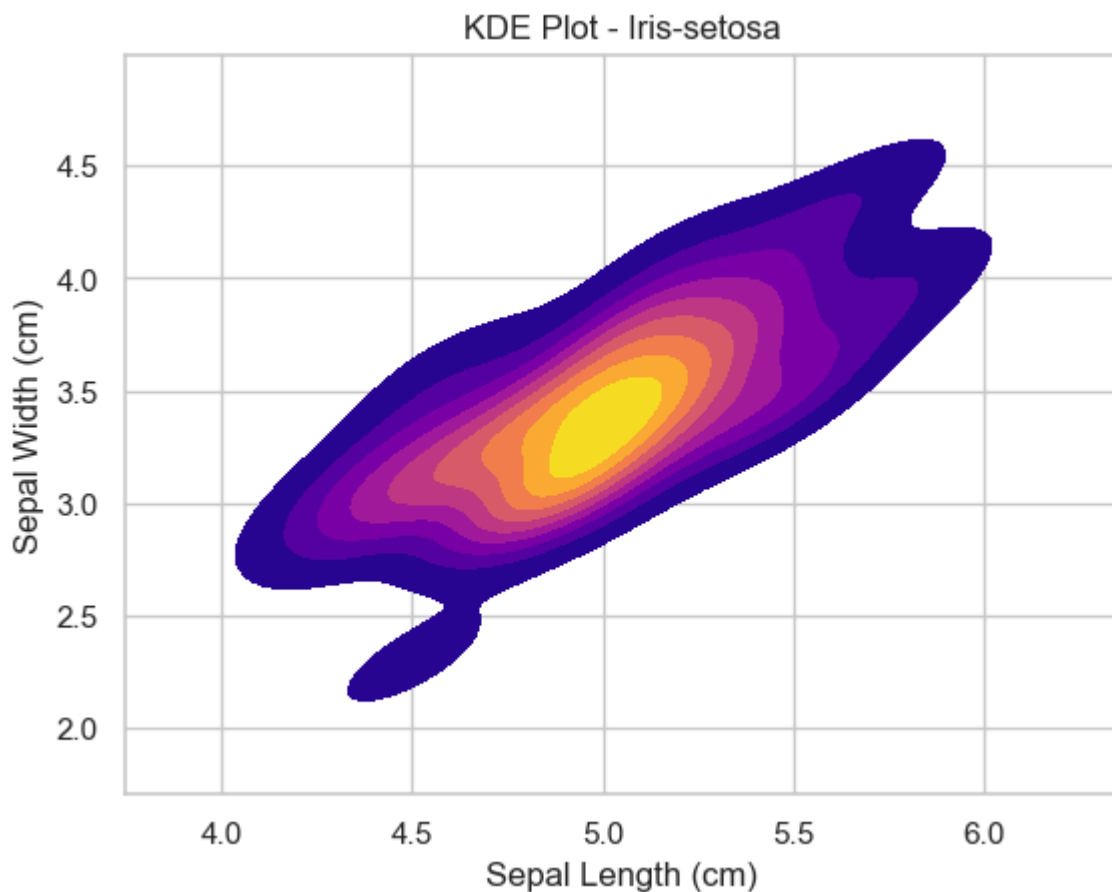
```
sns.kdeplot(data=sub[[x='SepalLengthCm',y='SepalWidthCm']],cmap="plasma", sha
de=True, shade_lowest=False)
```

SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?

```
In [45]: # Subset only 'Iris-setosa'
sub = iris[iris['Species'] == 'Iris-setosa']

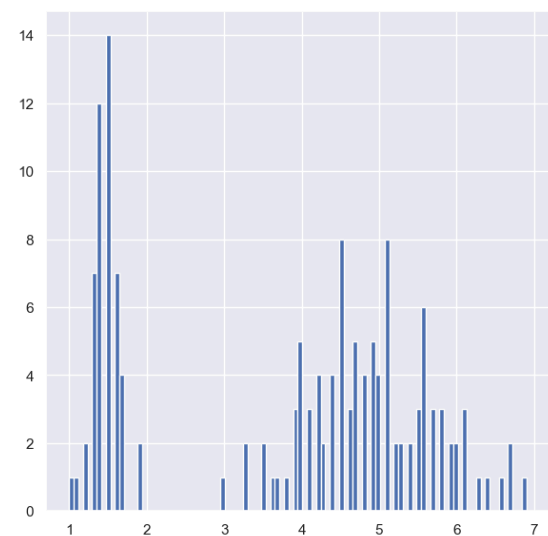
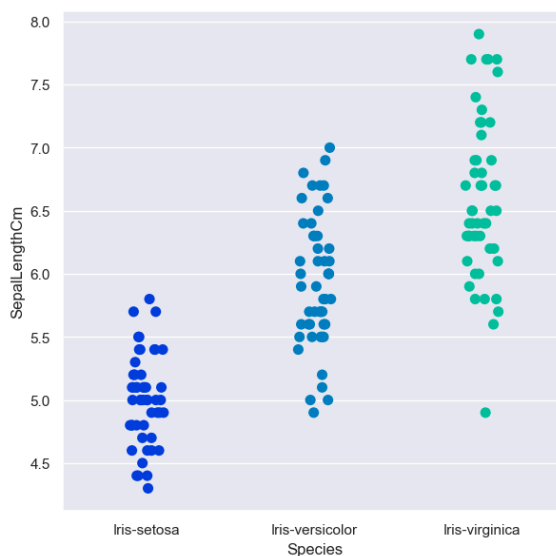
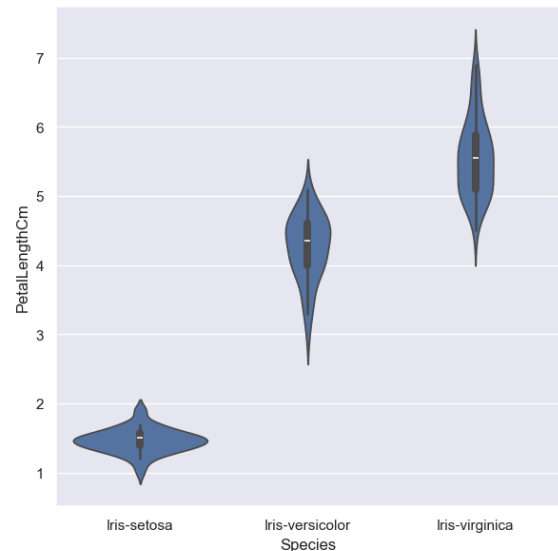
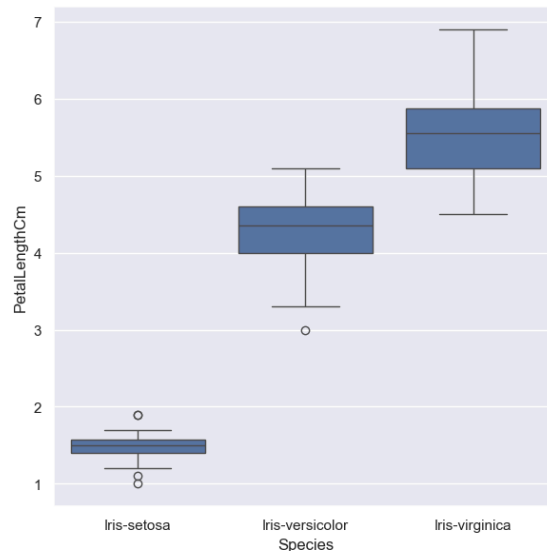
# KDE plot
sns.kdeplot(
    x=sub['SepalLengthCm'],
    y=sub['SepalWidthCm'],
    cmap="plasma",
    shade=True,          # use instead of shade=True
    shade_lowest=False,
)

plt.title('KDE Plot - Iris-setosa')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
```



```
In [46]: sns.set_style('darkgrid')
f,axes=plt.subplots(2,2,figsize=(15,15))

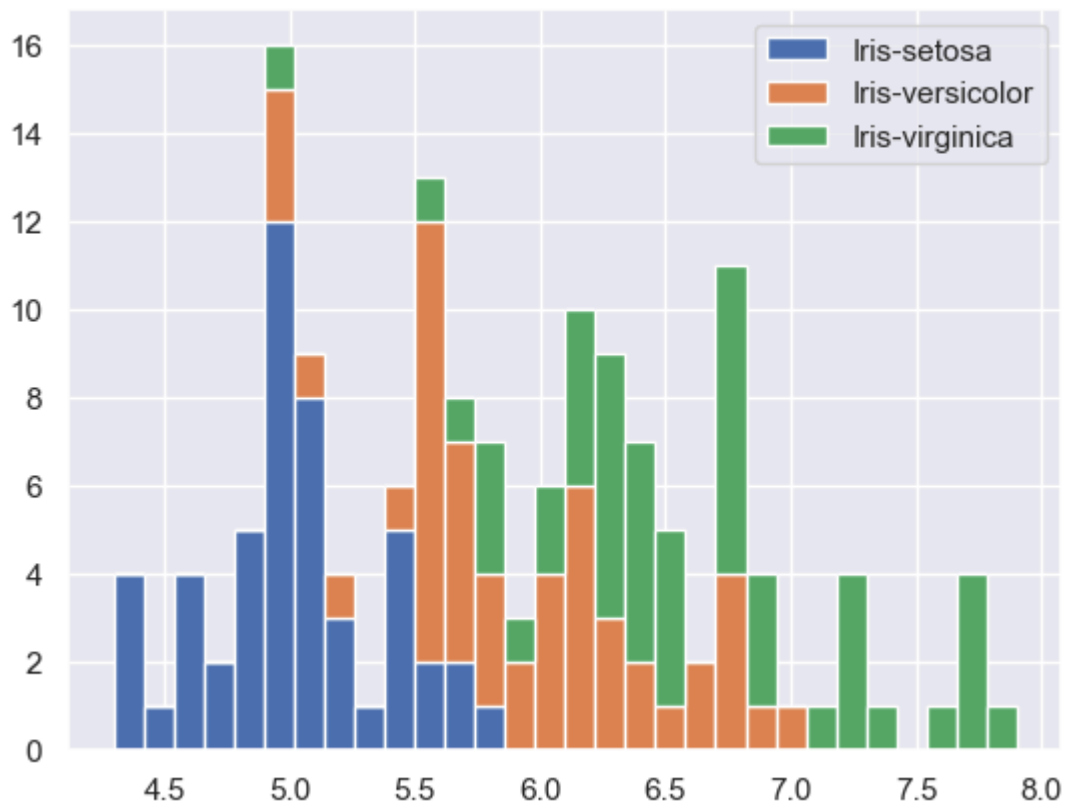
k1=sns.boxplot(x="Species", y="PetalLengthCm", data=iris,ax=axes[0,0])
k2=sns.violinplot(x='Species',y='PetalLengthCm',data=iris,ax=axes[0,1])
k3=sns.stripplot(x='Species',y='SepalLengthCm',data=iris,jitter=True,edgecolor='
#axes[1,1].hist(iris.hist,bin=10)
axes[1,1].hist(iris.PetalLengthCm,bins=100)
#k2.set(xlim=(-1,0.8))
plt.show() #dashboard
```



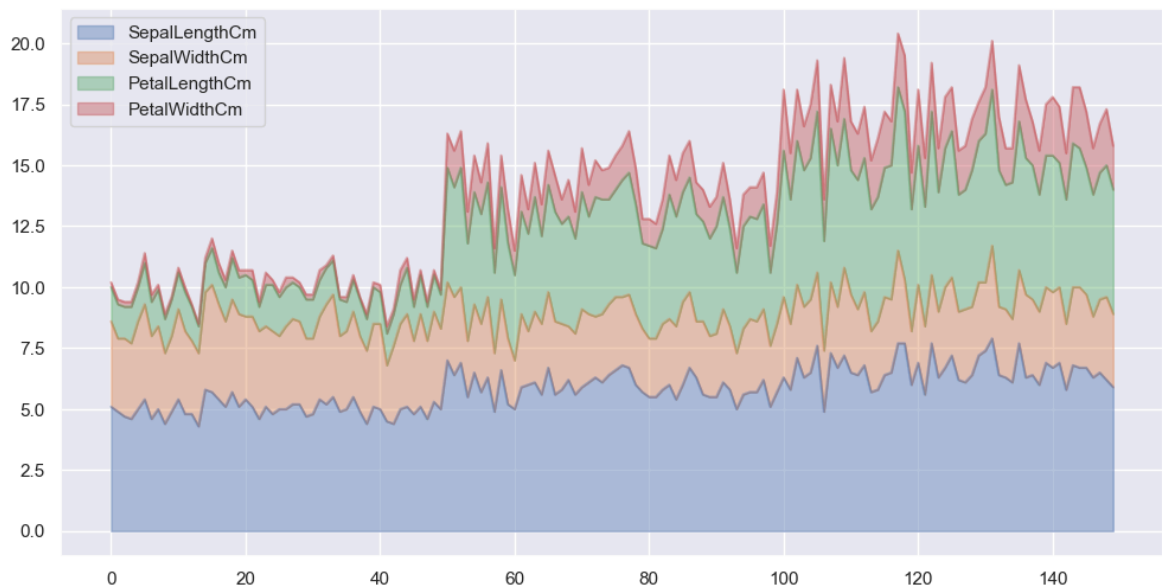
```
In [47]: iris['Species'] = iris['Species'].astype('category') # stacked histogram
```

```
In [49]: list1=list()
mylabels=list()
for gen in iris.Species.cat.categories:
    list1.append(iris[iris.Species==gen].SepalLengthCm)
    mylabels.append(gen)

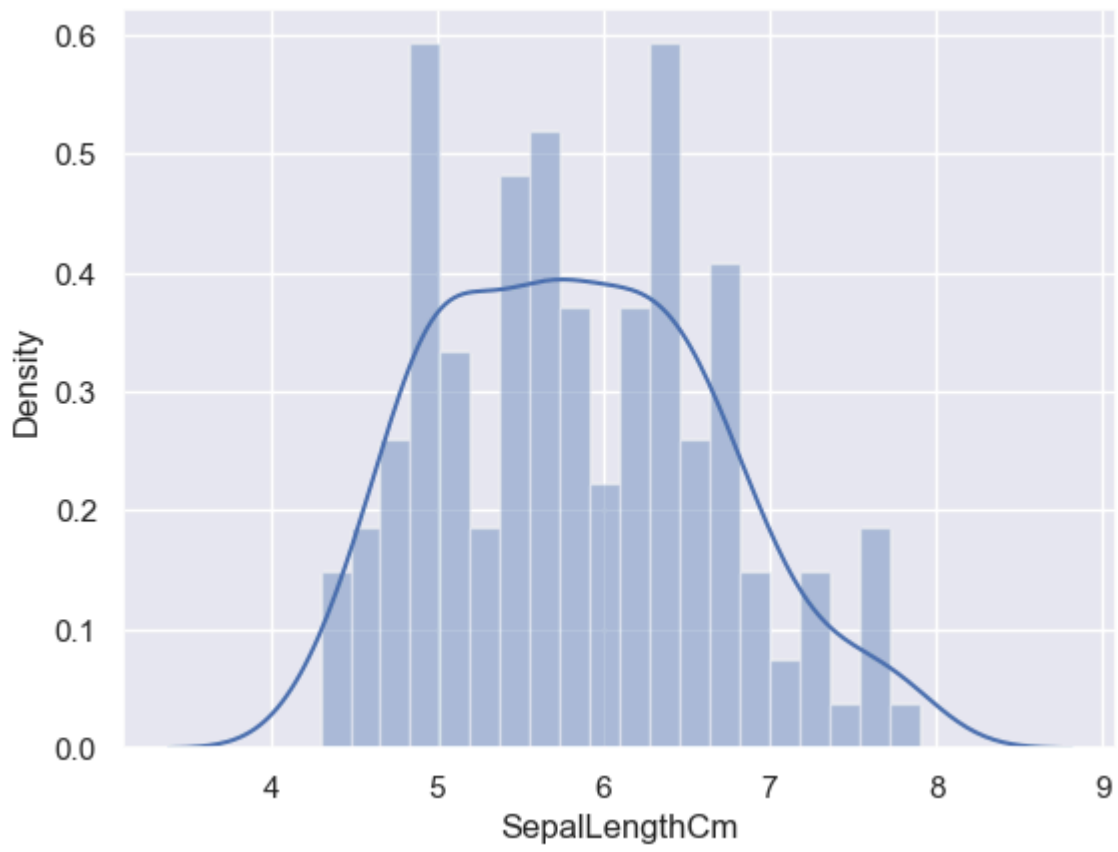
h=plt.hist(list1,bins=30,stacked=True,rwidth=1,label=mylabels)
plt.legend()
plt.show()
```



```
In [50]: iris.plot.area(y=['SepalLengthCm','SepalWidthCm','PetalLengthCm','PetalWidthCm'])  
plt.show() #area plot
```



```
In [51]: sns.distplot(iris['SepalLengthCm'],kde=True,bins=20);  
plt.show() #Dist plot
```



In []: